



Tesina 2

Analisi Algoritmo Genetico come Feature Selection

Gruppo A: *Gianmarco Luongo, Giada Pietrocola, Aurora Pisa, Giovanni Sivo*

Corso di Intelligenza Artificiale A.A. 2024/2025
Università degli Studi di Cassino e del Lazio Meridionale, Italia

Introduzione

L'uso degli **Algoritmi Genetici (GA)** nella feature selection rappresenta un campo cruciale per l'ottimizzazione e l'efficacia delle analisi di dati complessi. Nel contesto dello studio dell'Alzheimer, il dataset DARWIN fornisce un insieme ricco di caratteristiche di scrittura a mano, offrendo un terreno fertile per l'applicazione e l'analisi approfondita di tali algoritmi. Questa ricerca si propone di esplorare sistematicamente l'impatto dei parametri degli Algoritmi Genetici sulla convergenza, stabilità delle soluzioni, velocità di esecuzione e robustezza della selezione delle feature.

Metodologie

Codifica del cromosoma Per la rappresentazione delle soluzioni nell'Algoritmo Genetico, è stata adottata una codifica binaria del cromosoma, in cui ogni gene corrisponde a una feature del dataset **DARWIN**. Ogni cromosoma è dunque una stringa di bit di lunghezza pari al numero totale di feature disponibili, dove il valore 1 indica la selezione della feature corrispondente e il valore 0 ne indica l'esclusione. La scelta della codifica binaria è motivata dalla sua interpretabilità e dalla facilità con cui gli operatori di crossover e mutazione possono essere applicati per generare nuove soluzioni senza introdurre vincoli strutturali complessi.

Funzione di fitness La funzione di fitness è stata progettata per bilanciare l'importanza predittiva delle feature con la loro ridondanza interna, al fine di selezionare il sottoinsieme più informativo e meno ridondante. Per ottimizzare le prestazioni computazionali, le correlazioni tra le feature e il target, così come quelle tra le feature stesse, sono state precalcolate prima dell'esecuzione dell'algoritmo genetico. Questa strategia riduce significativamente la complessità computazionale temporale, evitando il ricalcolo delle stesse informazioni a ogni valutazione della fitness.

Il punteggio di fitness di ciascun cromosoma viene determinato considerando tre fattori principali: (i) la somma delle correlazioni assolute tra le feature selezionate e il target, che misura il potenziale informativo del sottoinsieme scelto, (ii) una penalità per la ridondanza, calcolata come somma delle correlazioni tra le feature selezionate, in modo da evitare insiemi di feature fortemente correlate tra loro, e (iii) un termine di penalizzazione per il numero di feature selezionate, che favorisce soluzioni compatte. La funzione di fitness è quindi definita come:

$$F(S) = \frac{|S|}{450} + P(S) - C(S) \quad (1)$$

dove:

- S è l'insieme delle feature selezionate dal cromosoma;
- $|S|$ è il numero di feature selezionate;

- $C(S)$ è il contributo informativo del sottoinsieme selezionato, calcolato come:

$$C(S) = \frac{1}{|S|} \sum_{i \in S} |\rho_{iY}| \quad (2)$$

dove ρ_{iY} rappresenta la correlazione tra la feature i e il target Y ;

- $P(S)$ è la penalità per la ridondanza tra le feature selezionate, definita come:

$$P(S) = \frac{1}{\binom{|S|}{2}} \sum_{i \neq j \in S} |\rho_{ij}| \quad (3)$$

dove ρ_{ij} è la correlazione tra le feature i e j , e il denominatore rappresenta il numero totale di coppie di feature selezionate.

Abbiamo utilizzato due tipi di correlazione: la **correlazione punto-biserial** per misurare la relazione tra ogni feature e il target binario, in quanto adatta a variabili continue rispetto a una variabile dicotomica, e la **correlazione di Pearson** per quantificare la dipendenza lineare tra le feature, permettendo di valutare la ridondanza informativa all'interno del sottoinsieme selezionato.

L'obiettivo dell'algoritmo genetico è *minimizzare* questa funzione di fitness, in modo da trovare il sottoinsieme di feature che massimizza l'informatività rispetto al target riducendo al minimo la ridondanza tra le feature selezionate.

Strategia di esecuzione Per garantire una valutazione robusta delle prestazioni dell'algoritmo genetico ed evitare risultati influenzati dalla casualità della popolazione iniziale e degli operatori genetici, sono state effettuate **30 run indipendenti**. Ogni run è stata eseguita utilizzando un *seed* incrementale, in modo che il valore del seed aumentasse di uno a ogni nuova esecuzione, assicurando così una variazione controllata nelle condizioni iniziali. I risultati finali sono stati analizzati aggregando le metriche di performance su tutte le esecuzioni, e i grafici risultanti rappresentano la media dei valori ottenuti nelle 30 run, permettendo di valutare la tendenza generale dell'algoritmo senza essere influenzati da singole variazioni casuali.

Diversità della popolazione Per monitorare la *diversità della popolazione* durante l'evoluzione

dell'algoritmo genetico, è stata calcolata la *distanza di Hamming media* tra i cromosomi presenti nella popolazione a ogni generazione. La distanza di Hamming misura il numero di posizioni in cui due cromosomi differiscono, fornendo un'indicazione del grado di variabilità all'interno della popolazione.

Il calcolo è stato effettuato considerando tutte le coppie di individui nella popolazione e determinando la distanza di Hamming per ciascuna coppia. La diversità media della popolazione è stata quindi definita come:

$$D = \frac{1}{N_{\text{coppie}} \cdot L} \sum_{i=1}^N \sum_{j=i+1}^N H(C_i, C_j) \quad (4)$$

dove:

- N è il numero totale di individui nella popolazione;
- L è la lunghezza del cromosoma (numero di feature);
- $H(C_i, C_j)$ rappresenta la distanza di Hamming tra il cromosoma C_i e il cromosoma C_j , ovvero il numero di posizioni in cui differiscono;
- $N_{\text{coppie}} = \frac{N(N-1)}{2}$ è il numero totale di coppie di individui considerate nel calcolo.

Questo approccio permette di valutare la *variazione genetica* all'interno della popolazione e identificare eventuali fenomeni di convergenza prematura, in cui la popolazione diventa troppo omogenea, riducendo l'efficacia dell'algoritmo nell'esplorazione dello spazio delle soluzioni.

Risultati

Dimensione Popolazione L'analisi dell'effetto della dimensione della popolazione sull'evoluzione dell'algoritmo genetico è stata condotta mantenendo invariati gli altri parametri e testando quattro valori di popolazione: 20, 50, 100 e 500. I risultati sono stati valutati in termini di velocità di convergenza, stabilità della selezione delle feature e costo computazionale.

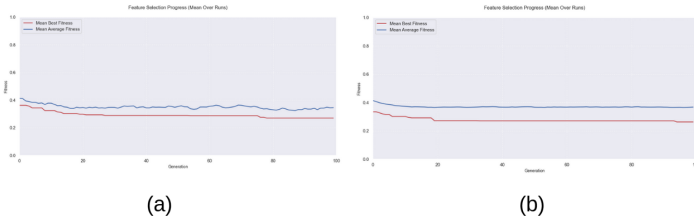


Figura 1: Grafici dell'andamento della fitness per una run per diversi valori di dimensione di popolazione: (a) popolazione=20, (b) popolazione=500

Dal grafico dell'andamento della fitness per generazione, si osserva che aumentando la dimensione della popolazione, la velocità di convergenza inizialmente rallenta. Per le dimensioni più piccole della popolazione (20 e 50), la best fitness tende a migliorare più rapidamente nelle prime generazioni, ma la convergenza diventa instabile a causa della scarsa diversità genetica. Al contrario, con una popolazione più grande (100 e 500), la convergenza è più lenta all'inizio, ma le soluzioni risultano più stabili nelle generazioni successive, evidenziando una maggiore robustezza dell'algoritmo.

L'analisi della frequenza di selezione delle feature mostra che con popolazioni piccole, la selezione delle feature è più concentrata, mentre con popolazioni più grandi le feature tendono a essere selezionate in modo più diversificato, indicando una maggiore esplorazione dello spazio delle soluzioni. Questo comportamento suggerisce che una popolazione maggiore consenta una ricerca più ampia e un miglior bilanciamento tra diversità e qualità delle soluzioni, ma potrebbe comportare un rallentamento nel trovare il miglior insieme di feature.

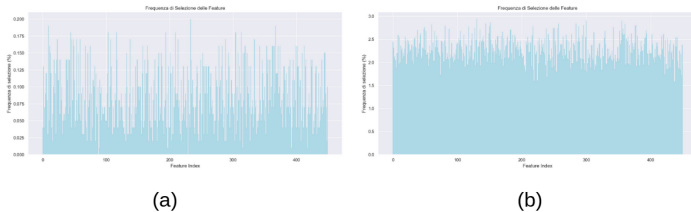


Figura 2: Grafici della frequenza di selezione delle features per una run diversi valori di dimensione di popolazione: (a) popolazione=20, (b) popolazione=500

Dal punto di vista computazionale, si nota che l'incremento della dimensione della popolazione

comporta un costo computazionale crescente. Le popolazioni più piccole permettono di completare le generazioni rapidamente, mentre per una popolazione di 500 individui il tempo di esecuzione risulta circa 100 volte superiore rispetto a una popolazione di 20 individui. In particolare, per una popolazione di 20 individui, il tempo medio per run è di circa 0.7 secondi, mentre per una popolazione di 500 individui il tempo medio sale a circa 70 secondi, rendendo l'algoritmo significativamente più oneroso.

Complessivamente, i risultati suggeriscono che una popolazione di 100 individui rappresenti un buon compromesso tra velocità di convergenza, stabilità della selezione delle feature e costo computazionale, permettendo di ottenere soluzioni affidabili senza un eccessivo incremento del tempo di esecuzione.

Crossover rate L'analisi dei risultati ottenuti variando il crossover rate tra i valori 0.6, 0.7, 0.8, 0.9 ha permesso di valutare il suo effetto sulla convergenza dell'algoritmo genetico e sulla qualità delle soluzioni finali.

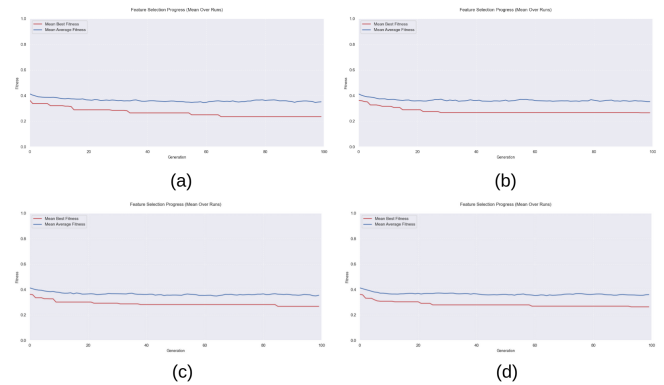


Figura 3: Grafici dell'andamento della fitness per una run per diversi valori di crossover rate: (a) crossover rate=0.6, (b) crossover rate=0.7, (c) crossover rate=0.8, (d) crossover rate=0.9

Dal grafico dell'andamento della fitness per generazione, si osserva che aumentando il crossover rate, la best fitness tende inizialmente a migliorare più rapidamente, indicando una maggiore capacità esplorativa dell'algoritmo nelle prime generazioni.

L'average fitness, invece, mostra un comportamento più controllato per crossover rate intermedi (0.7-0.8), mentre per valori estremi (0.6 e 0.9)

si evidenziano rispettivamente una convergenza più lenta e una maggiore variabilità tra le run. Questo suggerisce che valori eccessivamente alti di crossover possano aumentare il rischio di perdita di buone soluzioni a causa di una eccessiva ricombinazione.

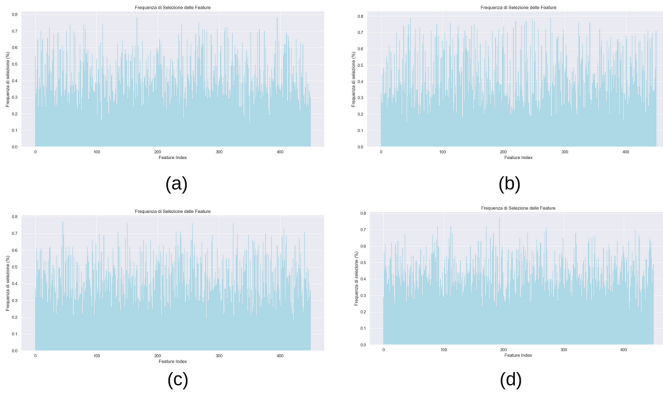


Figura 4: Grafici della frequenza di selezione delle features per una run diversi valori di crossover rate: (a) crossover rate=0.6, (b) crossover rate=0.7, (c) crossover rate=0.8, (d) crossover rate=0.9

L'analisi della frequenza di selezione delle feature evidenzia che per valori di crossover più alti, le feature vengono selezionate con una distribuzione meno concentrata, suggerendo una maggiore esplorazione dello spazio delle soluzioni. Tuttavia, quando il crossover rate è troppo basso, la selezione delle feature tende a stabilizzarsi rapidamente su un insieme ristretto, indicando una minore diversità delle soluzioni.

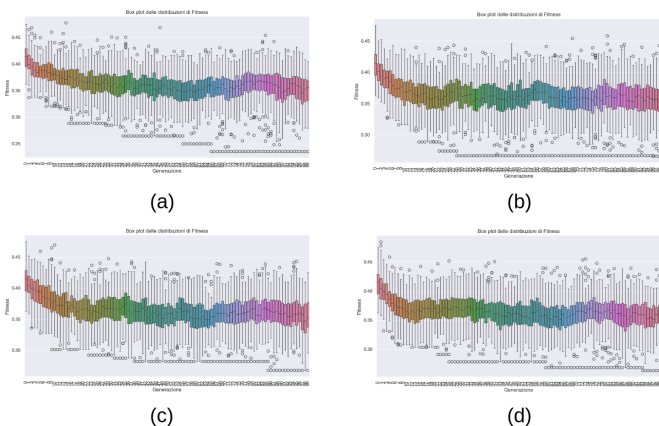


Figura 5: Box plot della fitness per generazione per una run per diversi valori di crossover rate: (a) crossover rate=0.6, (b) crossover rate=0.7, (c) crossover rate=0.8, (d) crossover rate=0.9

Dal box plot della fitness per generazione, emerge che crossover rate intermedi garantiscono una maggiore stabilità delle soluzioni rispetto ai valori estremi, con una minore varianza nei risultati finali. Infine, la diversità della popolazione per generazione mostra un andamento coerente con le osservazioni precedenti: per valori di crossover più bassi la diversità cala rapidamente, mentre per valori alti persiste più a lungo, ritardando la convergenza.

Complessivamente, i risultati suggeriscono che un crossover rate compreso tra 0.7 e 0.8 rappresenti un buon compromesso tra esplorazione e convergenza, garantendo soluzioni stabili e di qualità senza compromettere la diversità genetica della popolazione.

Mutation rate L'analisi dell'effetto del tasso di mutazione è stata condotta mantenendo invariati gli altri parametri e testando quattro valori di mutazione: 0.01, 0.05, 0.1 e 0.15. I risultati sono stati valutati attraverso il grafico dell'andamento della fitness per generazione, il grafico della frequenza di selezione delle feature, il box plot della fitness per generazione e la diversità della popolazione per generazione.

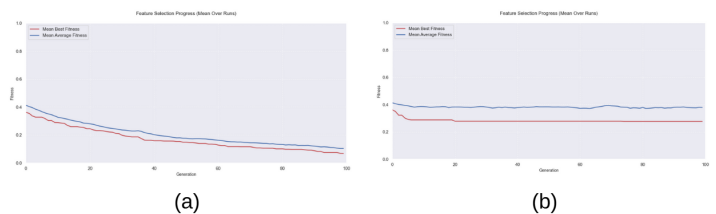


Figura 6: Grafici dell'andamento della fitness per una run per diversi valori di mutation rate: (a) mutation rate=0.01, (b) mutation rate=0.15

Dal grafico dell'andamento della fitness per generazione, si osserva che per valori di mutazione molto bassi (0.01) l'algoritmo converge più lentamente, ma verso soluzioni di qualità superiore. Questo suggerisce che un tasso di mutazione basso favorisca un'esplorazione più controllata dello spazio delle feature, consentendo all'algoritmo di individuare combinazioni più ottimali. Al contrario, per valori di mutazione elevati (0.15), l'algoritmo converge molto rapidamente ma verso soluzioni subottimali, indicando che una mutazione eccessiva introdu-

ce troppa variabilità, impedendo la raffinazione delle soluzioni.

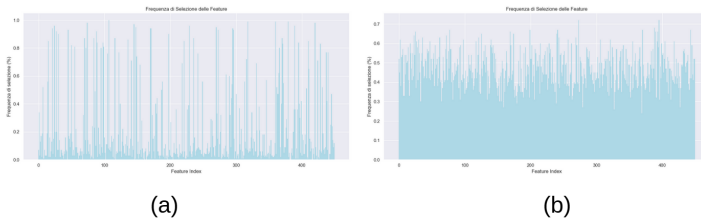


Figura 7: Grafici della frequenza di selezione delle feature per una run per diversi valori di mutation rate: (a) mutation rate=0.01, (b) mutation rate=0.15

L'analisi della frequenza di selezione delle feature mostra che con tassi di mutazione bassi (0.01), le feature selezionate risultano più stabili tra le diverse run, con una maggiore coerenza nella scelta delle feature più rilevanti. Con l'aumento del tasso di mutazione, la distribuzione delle feature selezionate diventa più dispersa, evidenziando un'esplorazione più ampia ma meno focalizzata.

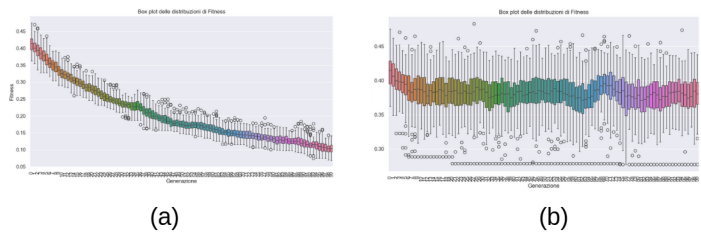


Figura 8: Box plot della fitness per generazione per una run per diversi valori di mutation rate: (a) mutation rate=0.01, (b) mutation rate=0.15

Il box plot della fitness per generazione evidenzia che per tassi di mutazione elevati (0.15), la variabilità tra le run è ridotta, poiché l'algoritmo converge rapidamente verso soluzioni simili, anche se subottimali. Al contrario, con tassi di mutazione bassi (0.01), la variabilità è maggiore, ma il valore finale della best fitness tende a essere migliore, segnalando un miglior processo di ottimizzazione.

La diversità della popolazione per generazione mostra un andamento coerente con queste osservazioni: per tassi di mutazione bassi, la diversità diminuisce più lentamente, favorendo una ricerca più estesa, mentre per tassi elevati la diversità si riduce molto rapidamente, portando a una convergenza prematura. In particolare, si

nota che per il caso estremo di mutazione 0.01, alla fine dell'algoritmo la diversità della popolazione si attesta intorno a 0.16, indicando che la popolazione si è fortemente concentrata su poche soluzioni ottimali. Al contrario, per il caso di mutazione 0.15, la diversità finale rimane sorprendentemente alta a 0.49, molto vicina al valore iniziale di 0.5, quando la popolazione era ancora casuale. Questo suggerisce che un'elevata mutazione impedisce alla popolazione di convergere verso un sottoinsieme ristretto di soluzioni, mantenendo un'elevata variabilità ma senza un reale miglioramento della fitness.

Complessivamente, i risultati suggeriscono che un tasso di mutazione intorno a 0.05 - 0.1 rappresenti un buon compromesso tra velocità di convergenza e qualità delle soluzioni, evitando sia la convergenza prematura di tassi troppo elevati, sia la lentezza nell'ottimizzazione di tassi troppo bassi.

Criterio di arresto L'ultimo scenario analizza l'effetto della variazione del criterio di arresto dell'algoritmo genetico, mantenendo invariati gli altri parametri.

Dal grafico dell'andamento della fitness per generazione, si nota che con 50 generazioni l'algoritmo tende a interrompersi troppo presto, senza raggiungere una fase di convergenza stabile. Questo porta a soluzioni subottimali, con una best fitness non ancora stabilizzata. Con 100 generazioni, si osserva un miglioramento significativo, mentre 200 generazioni permettono una ricerca più approfondita, ma con un incremento dei tempi di esecuzione.

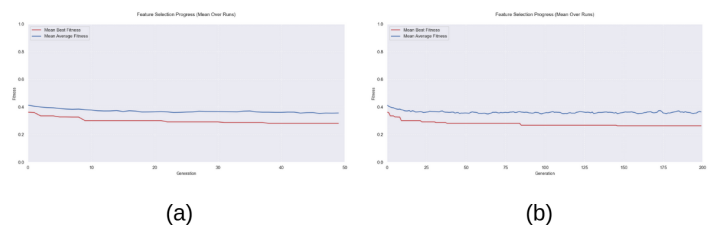


Figura 9: Grafici dell'andamento della fitness per una run per diversi valori di generazioni massime: (a) max generations=50, (b) max generations=100

L'introduzione di un criterio di arresto basato sulla convergenza migliora l'efficienza dell'algoritmo, riducendo il numero di generazioni

necessarie senza compromettere la qualità della soluzione.

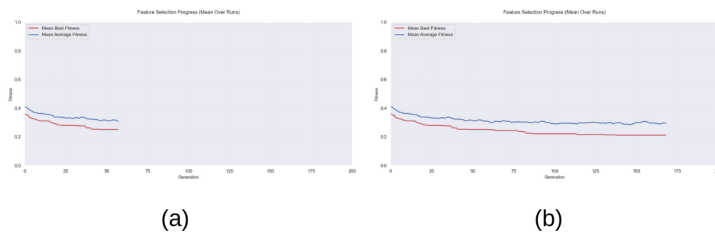


Figura 10: Grafici dell'andamento della fitness per una run per diversi valori di soglie di generazioni senza miglioramento: (a) soglia = 10, tolleranza = $1e-6$, (b) soglia = 30, tolleranza = $1e-6$

Con soglie di 10 generazioni senza miglioramento, l'algoritmo si arresta prematuramente nella maggior parte delle run, risultando in soluzioni meno stabili. Con soglie di 20 e 30 generazioni, si osserva un comportamento più bilanciato, dove l'algoritmo ha il tempo di esplorare lo spazio delle soluzioni prima di fermarsi. Tuttavia, l'uso dell'elitismo ha ridotto significativamente l'impatto delle diverse tolleranze sulla variazione della fitness. Poiché gli individui migliori vengono preservati in ogni generazione, l'algoritmo tende a convergere in modo simile indipendentemente dal valore della tolleranza scelto. Questo significa che, nella pratica, la scelta della tolleranza ha avuto un'influenza minore rispetto ad altri parametri come il numero massimo di generazioni o la soglia di stagnazione. Complessivamente, i risultati suggeriscono che un compromesso ottimale si ottiene con circa 100 generazioni fisse oppure con una soglia di 20 generazioni senza miglioramento. Questo permette di ottenere soluzioni stabili, mantenendo sotto controllo i tempi di esecuzione.